Scalable Microservice Deployment on Amazon EKS: A Terraform-Driven Kubernetes Architecture on AWS

1. Abstract

Modern applications demand high availability, scalability, and rapid deployment. Traditional hosting environments require organizations to manually configure physical or virtual servers, manage infrastructure, and handle sudden traffic spikes, resulting in increased operational complexity, cost, and risk of downtime. As applications grow in scale and complexity, monolithic architectures struggle to meet these demands efficiently.

This project, "Scalable Microservice Deployment on Amazon EKS: A Terraform-Driven Kubernetes Architecture on AWS," addresses these challenges by designing and implementing a cloud-native microservice deployment architecture. The project leverages Amazon Elastic Kubernetes Service (EKS) for managed container orchestration and Terraform as the Infrastructure-as-Code (IaC) tool to provision and manage the complete AWS infrastructure in a consistent and repeatable manner.

The purpose of this project is to demonstrate how microservices can be efficiently deployed, scaled, and managed using modern DevOps practices on AWS. The expected outcome is a secure, highly available, fault-tolerant, and automated Kubernetes deployment blueprint that eliminates manual infrastructure management and supports rapid application scaling, rolling updates, and automatic failure recovery.

2. Introduction

The rapid evolution of cloud computing has transformed how organizations build and deliver software. Businesses increasingly adopt microservice architectures, where large applications are broken into smaller, independently deployable services. While microservices offer flexibility and faster development cycles, managing hundreds of containerized services introduces significant orchestration complexity.

Kubernetes has emerged as the industry-standard container orchestration platform, providing automated deployment, scaling, and self-healing capabilities. However, setting up and maintaining a Kubernetes cluster manually demands deep technical expertise. Amazon EKS solves this by providing a fully managed Kubernetes control plane, allowing teams to

focus on application workloads rather than infrastructure management.

Terraform, an open-source Infrastructure-as-Code tool, allows cloud infrastructure to be defined in configuration files that are version-controlled, reproducible, and auditable. Together, Amazon EKS and Terraform form a powerful combination for deploying production-grade microservice architectures on AWS. This project is important because it bridges the gap between theoretical cloud concepts and real-world enterprise deployment practices, providing a practical blueprint for scalable microservice delivery.

3. Key Points of the Project

- Terraform-based Infrastructure Provisioning - VPC, subnets, NAT gateways, route tables, security groups, EKS cluster, and managed node groups are all defined and provisioned through Terraform configuration files.
- Amazon EKS Managed Kubernetes - The Kubernetes control plane is fully managed by AWS, reducing operational overhead and ensuring high availability.
- Multi-Availability Zone (Multi-AZ) Architecture - Worker nodes and workloads are distributed across multiple AWS Availability Zones for fault tolerance.
- Amazon ECR for Container Image Management - Microservice container images are stored in a private Elastic Container Registry for secure and controlled image distribution.
- AWS Elastic Load Balancing - Application Load Balancer (ALB) for HTTP/HTTPS and Network Load Balancer (NLB) for TCP/UDP traffic distribution across healthy pods.
- IAM and Kubernetes RBAC Security - AWS IAM is integrated with Kubernetes RBAC to enforce granular access policies at both the AWS and cluster levels.
- Cluster Autoscaler - Dynamically adjusts worker node capacity based on real-time workload demands, scaling up or down as required.
- Monitoring & Observability - Integration with Amazon CloudWatch, Prometheus, and Grafana for metrics collection, logging, alerting, and dashboards.
- Rolling Updates and Self-Healing - Kubernetes supports rolling updates with zero downtime and automatically restarts failed pods via health checks.
- Infrastructure-as-Code Best Practices - All infrastructure is version-controlled, auditable, and repeatable, supporting modern DevOps and CI/CD workflows.

4. Advantages & Limitations

Advantages:

- Scalability: Dynamically scales both application pods and worker nodes based on demand, ensuring consistent performance during traffic spikes.
- High Availability: Multi-AZ deployment ensures the application remains operational even if an entire Availability Zone experiences an outage.
- Reduced Operational Overhead: Amazon EKS manages the Kubernetes control plane, eliminating the need for manual patching or maintenance of master nodes.
- Repeatability and Consistency: Terraform ensures infrastructure can be reproduced identically across development, staging, and production environments.
- Cost Efficiency: Cluster Autoscaler terminates idle worker nodes, ensuring compute resources are only consumed when required.
- Security: Private networking, security groups, AWS IAM, and Kubernetes RBAC enforce a defense-in-depth security model for the entire stack.
- Faster Deployment: Containerized microservices can be deployed, updated, and rolled back quickly using Kubernetes manifests.

Limitations:

- Complexity: Managing Kubernetes, Terraform state, IAM policies, and networking simultaneously requires significant expertise and a steep learning curve.
- Cost of EKS Control Plane: AWS charges a fixed hourly fee per EKS cluster regardless of workload size, which may be costly for small-scale projects.
- Terraform State Management: The Terraform state file must be securely stored (e.g., S3 with DynamoDB locking) to prevent corruption in team environments.
- Monitoring Setup Overhead: Configuring Prometheus, Grafana, and CloudWatch integrations adds initial setup complexity and ongoing maintenance.
- Vendor Lock-in: The architecture is tightly coupled to AWS services (EKS, ECR, ALB, IAM), making migration to other cloud providers non-trivial.

5. Components in the Diagram

- **Amazon VPC:**
  A logically isolated virtual network on AWS hosting all infrastructure components, divided into public and private subnets across multiple Availability Zones.
- **Public Subnets:**
  Host internet-facing resources such as the Application Load Balancer and NAT Gateways, accessible from the internet via the Internet Gateway.

- **Private Subnets:**

  Host the EKS worker nodes and application pods, isolated from direct internet access for enhanced security.

- **Internet Gateway:**

  Enables bidirectional communication between the VPC and the public internet for internet-facing resources.

- **NAT Gateway:**

  Allows private subnet resources (worker nodes) to initiate outbound internet connections without exposing them to inbound internet traffic.

- **Amazon EKS Cluster:**

  The managed Kubernetes control plane provided by AWS, responsible for scheduling workloads, managing cluster state, and exposing the Kubernetes API.

- **Managed Node Groups:**

  EC2 instances managed by AWS that serve as Kubernetes worker nodes, running application pods and distributed across multiple Availability Zones.

- **Kubernetes Pods:**

  The smallest deployable units in Kubernetes, each running one or more containers for a specific microservice.

- **Amazon ECR:**

  A private container image registry that stores Docker images for each microservice, from which Kubernetes pulls images during deployment.

- **Application Load Balancer (ALB):**

  Distributes incoming HTTP/HTTPS traffic from users across healthy application pods running in the cluster.

- **Network Load Balancer (NLB):**

  Handles TCP/UDP-level traffic routing for services requiring ultra-low latency or non-HTTP protocols.

- **AWS IAM:**

  Manages AWS-level identity and permissions, including roles for EKS worker nodes and service accounts for secure API access.

- **Kubernetes RBAC:**

  Controls what authenticated users and service accounts can do within the Kubernetes cluster, providing a layered security model alongside IAM.

- **Cluster Autoscaler:**

  Monitors pending pods and automatically scales worker nodes up or down based on real-time resource demand.

- **CloudWatch / Prometheus / Grafana:**

  Monitoring and observability tools for collecting metrics, aggregating logs, creating dashboards, and triggering alerts for infrastructure and application health.

6. Future Scope

- CI/CD Pipeline Integration: Integrating tools such as GitHub Actions, AWS CodePipeline, or Jenkins with the EKS cluster to enable fully automated build, test, and deployment workflows, reducing manual intervention.
- Service Mesh Implementation: Adopting AWS App Mesh or Istio to provide advanced traffic management, mutual TLS encryption between microservices, circuit breaking, and detailed service-to-service observability.
- GitOps-Based Deployment: Implementing GitOps workflows using ArgoCD or Flux, where the Git repository serves as the single source of truth for Kubernetes configurations, enabling automated and auditable deployments.
- Multi-Region Disaster Recovery: Extending to a multi-region active-passive or active-active configuration with Amazon Route 53 DNS-based failover to ensure business continuity during regional AWS outages.
- Advanced Cost Optimization: Incorporating AWS Spot Instances, Karpenter for intelligent node provisioning, and AWS Compute Optimizer right-sizing to further reduce infrastructure costs without sacrificing performance.

7. Conclusion

This project successfully demonstrates the design and implementation of a scalable, secure, and highly available microservice deployment architecture on Amazon Web Services using Amazon EKS and Terraform. By adopting a Terraform-driven Infrastructure-as-Code approach, the entire cloud infrastructure including the VPC, subnets, NAT gateways, EKS cluster, and managed node groups can be provisioned consistently, version-controlled, and reproduced across environments without manual configuration.

The use of Amazon EKS eliminates the complexity of managing a Kubernetes control plane, while Kubernetes features such as rolling updates, health checks, pod replicas, and the Cluster Autoscaler ensure that the application remains available, fault-tolerant, and

responsive to changing traffic demands. Security is enforced through a layered model combining private networking, AWS IAM, and Kubernetes RBAC.

Overall, this project provides practical, hands-on experience with industry-relevant technologies including AWS cloud infrastructure, Kubernetes orchestration, containerization, Infrastructure-as-Code, load balancing, auto-scaling, and observability. It establishes a robust foundation for deploying production-grade microservices and serves as a reusable blueprint that can be extended with CI/CD pipelines, service meshes, GitOps workflows, and multi-region disaster recovery in the future.